# Stochastic Processes

## Approximate Dynamic Programming and Reinforcement Learning

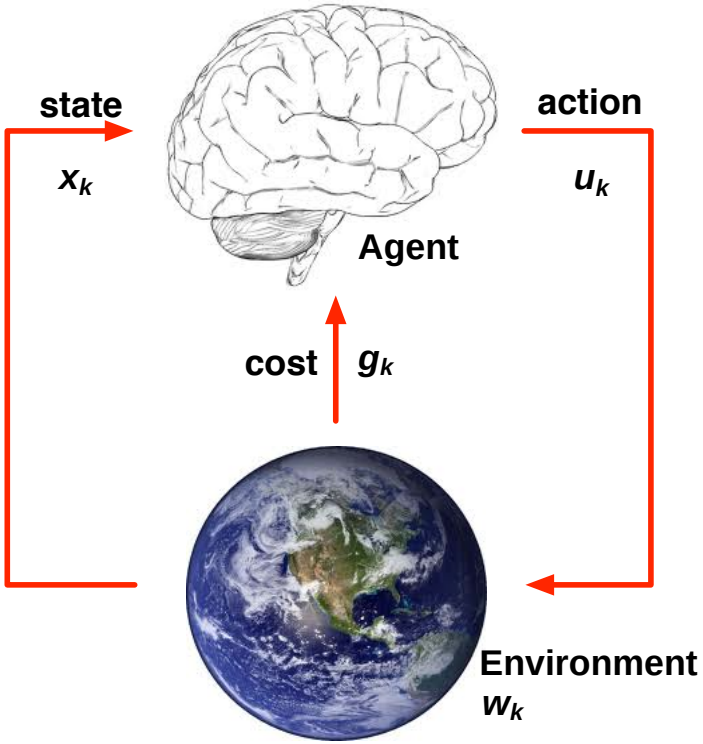
Jie Gong

Autumn, 2022

# Outline

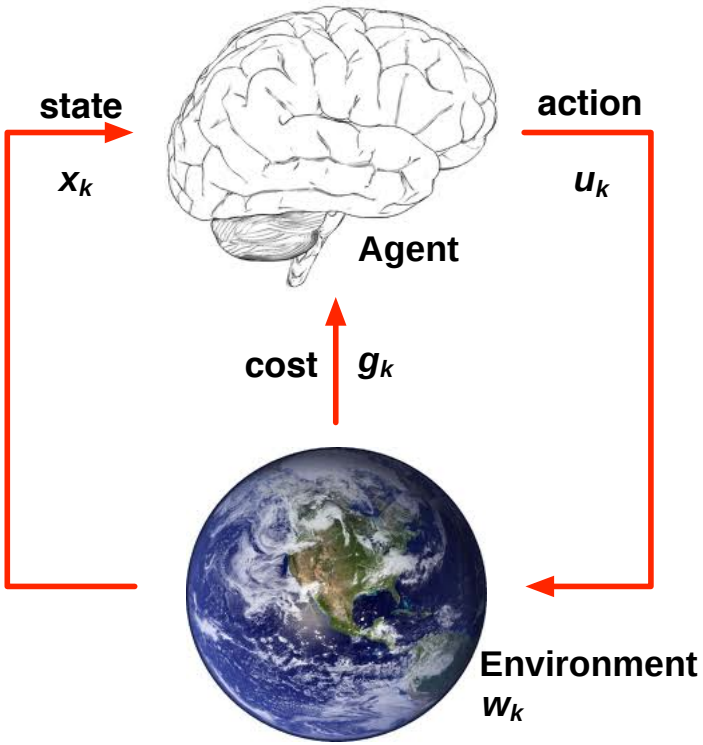

1. Introduction
2. Approximate Dynamic Programming
3. Reinforcement Learning

# Sequential Decision Problem



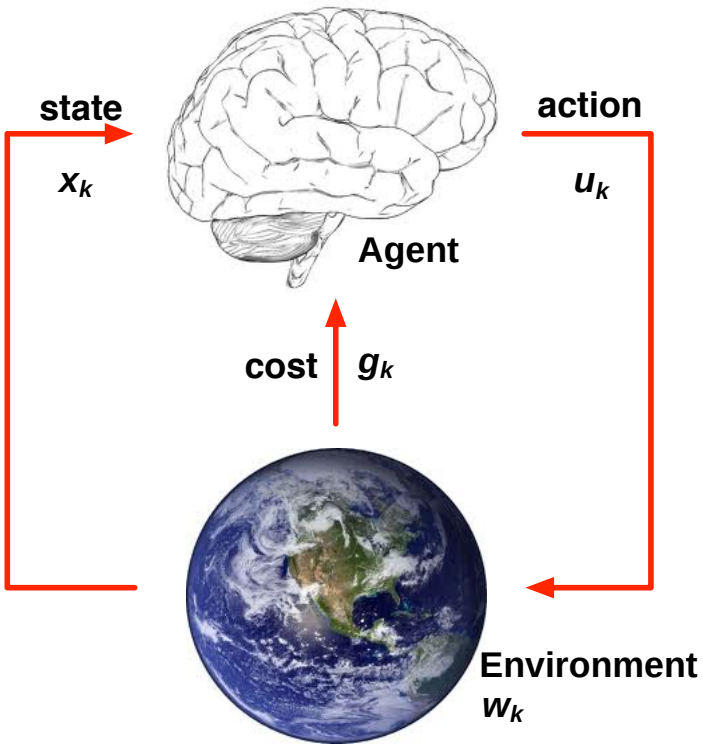

- Agent: take an action $u_k$ depending on observation
- Environment: generate a cost $g_k$ and transit to a new state $x_{k+1}$ with some randomness $w_k$

# MDP - Planning Problem




- A model of the environment is known ($g_k, f_k$, distribution of $w_k$)
- The agent optimizes its policy based on the model (without any external interaction), and carry out the policy

# RL - Learning Problem




- The environment is initially (partially) unknown
- The agent interacts with the environment and improves its policy
- Or the model is known but overcomplex (a.k.a. ADP)

# Outline


1. Introduction
2. Approximate Dynamic Programming
3. Reinforcement Learning

# Finite Horizon DP Problems

- System model

$$x_{k+1} = f_k(x_k, u_k, w_k), \qquad k = 0, 1, \ldots, N-1$$

- Objective

$$\min_{\pi} E\left\{\sum_{k=0}^{N-1} g_k(x_k, \mu_k(x_k), w_k) + g_N(x_N)\right\}$$

where $\pi = \{\mu_0, \mu_1, \ldots, \mu_{N-1}\}$

# DP Algorithm

- Start with

$$J_N(x_N) = g_N(x_N),$$

and go backwards using

$$J_k(x_k) = \min_{u_k} E_{w_k} \left\{ g_k(x_k, u_k, w_k) + J_{k+1}\left(f_k(x_k, u_k, w_k)\right) \right\},$$

where $k = 0, 1, \ldots, N-1$.

- The curse of dimensionality
  - Exponential growth of the computational and storage requirements as the number of state variables and control variables increases
  - Quick explosion of the number of states in combinatorial problems

# Value Function Approximation

- Use a policy computed from the DP equation where the optimal cost-to-go function $J_{k+1}$ is replaced by an approximation $\tilde{J}_{k+1}$
- Apply $\bar{\mu}_k(x_k)$, which attains the minimum in

$$\min_{u_k} E_{w_k}\left\{g_k(x_k, u_k, w_k) + \tilde{J}_{k+1}\left(f_k(x_k, u_k, w_k)\right)\right\},$$

- Different ways to compute $\tilde{J}_{k+1}$
  - Off-line approximation: The entire function $\tilde{J}_{k+1}$ is computed for every $k$, before the control process begins.
  - On-line approximation: Only the values $\tilde{J}_{k+1}(x_k + 1)$ at the relevant next states $x_k + 1$ are computed and used to compute $u_k$ just after the current state $x_k$ becomes known.
  - Simulation-based methods: Offline or online methods based on Monte-Carlo simulation

# Certainty Equivalent Control (CEC)

- Idea: Replace the stochastic problem with a deterministic problem
- At each time $k$, the future uncertain quantities are fixed at some "typical" values
- Off-line implementation:
  - Let $\{\mu_0^d, \ldots, \mu_{N-1}^d\}$ be an optimal controller obtained from the DP algorithm for the deterministic problem

$$\text{minimize} \quad \sum_{k=0}^{N-1} g_k(x_k, \mu_k(x_k), \bar{w}_k) + g_N(x_N)$$

$$\text{subject to} \quad x_{k+1} = f_k(x_k, \mu_k(x_k), \bar{w}_k),$$

  - The CEC applies at time $k$ the control $\mu_k^d(x_k)$

# CEC: On-line Implementation

- On-line implementation. At each time $k$:
  - Fix the $w_i, i \geq k$, at some $\bar{w}_i$. Solve the deterministic problem

$$\min_{u_i \in U_i} \sum_{i=k}^{N-1} g_i(x_i, u_i, \bar{w}_i) + g_N(x_N)$$

  for given $x_k$, where $x_{i+1} = f_i(x_i, u_i, \bar{w}_i)$
  - Use the first control $\bar{\mu}_k(x_k)$ in the optimal control sequence found.

# Limited Lookahead Policies

- One-step lookahead (1SL) policy
  - Use a policy computed from the DP equation where the optimal cost-to-go function $J_{k+1}$ is replaced by an approximation $\tilde{J}_{k+1}$
  - Apply $\bar{\mu}_k(x_k)$, which attains the minimum in

$$\min_{u_k} E_{w_k}\left\{g_k(x_k, u_k, w_k) + \tilde{J}_{k+1}\left(f_k(x_k, u_k, w_k)\right)\right\},$$

  - Given the approximated cost-to-go $\tilde{J}_k(x_k)$, the 1SL policy is implemented online
- Two-step lookahead policy
  - At each $k$ and $x_k$, use the control $\tilde{\mu}_k(x_k)$ attaining the minimum above, where the function $\tilde{J}_{k+1}$ is obtained using a 1SL approximation (solve a 2-step DP problem)

# Approximation Function Design

- Problem Approximation: Approximate the optimal cost-to-go with some cost derived from a related but simpler problem
- Parametric Cost-to-Go Approximation: Approximate the optimal cost-to-go with a function of a suitable parametric form, whose parameters are tuned by some heuristic or systematic scheme (Neuro-Dynamic Programming)
- Rollout Approach: Approximate the optimal cost-to-go with the cost of some suboptimal policy, which is calculated either analytically or by simulation

# Enforced decomposition

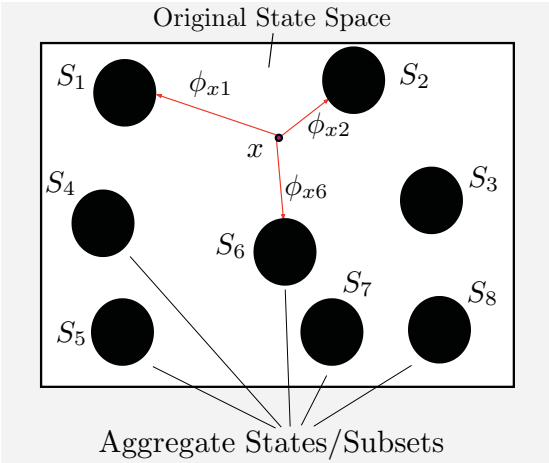
- The approximation is problem-dependent
- Example: Enforced decomposition for routing problem
  - Route $m$ vehicles that move over a graph. Each node has a "value". First vehicle that passes through the node collects its value.
  - Objective: max the total collected value, subject to the constraints on beginning nodes and destination nodes
  - Usually the 1-vehicle version of the problem is much simpler. This motivates an approximation obtained by solving single vehicle problems.
  - 1SL scheme: At time $k$ and state $x_k$ (position of vehicles and "collected value nodes"), consider all possible $k$-th moves by the vehicles, and at the resulting states we approximate the optimal value-to-go with the value collected by optimizing the vehicle routes one-at-a-time

# Aggregation

- Introduce "aggregate" states $S_1, \ldots, S_m$, viewed as the states of an "aggregate" system
- Define transition probabilities and costs of the aggregate system, by relating original system states with aggregate states (using so called "aggregation and disaggregation probabilities")
- Solve (exactly or approximately) the "aggregate" problem by any kind of method (including simulation-based)
- Use the optimal cost of the aggregate problem to approximate the optimal cost of each original problem state as a linear combination of the optimal aggregate state costs
- Discretization of continuous state spaces can be interpreted in terms of aggregation

# Representative Subsets

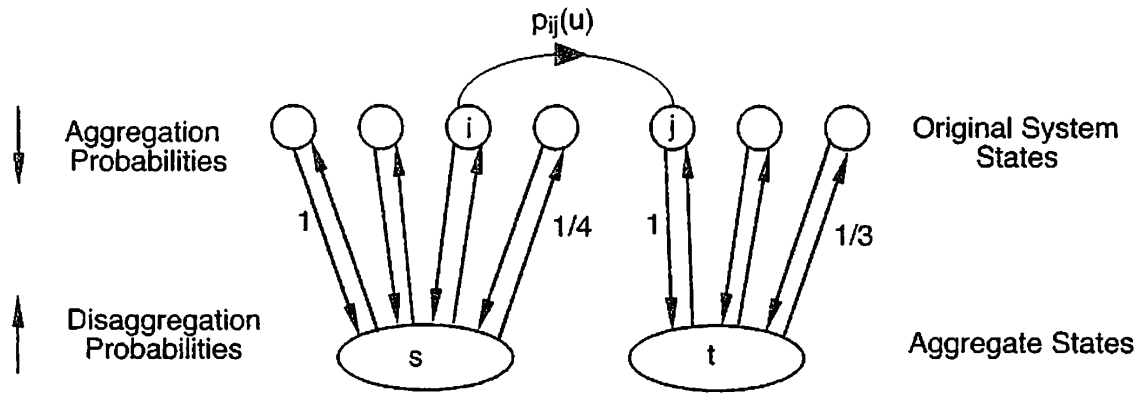
- The aggregate states $S_j$ are disjoint "representative" subsets of original system states



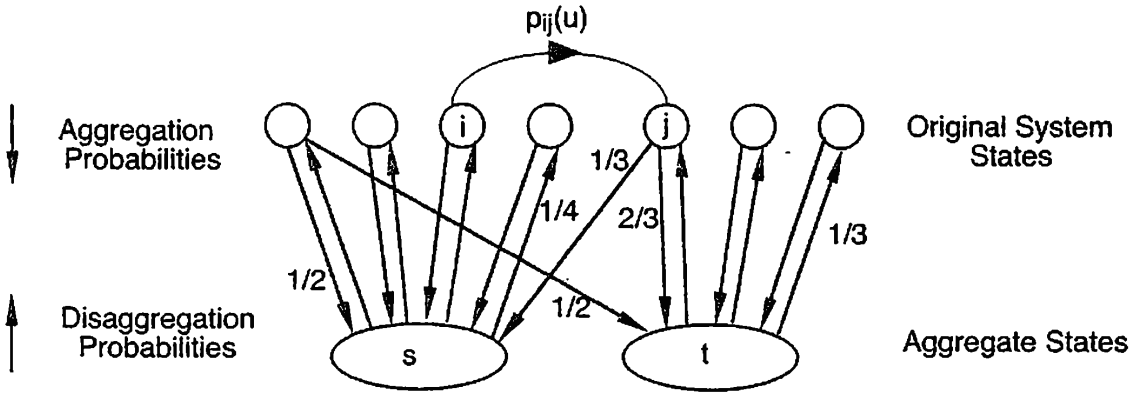

- Each $S_j$ is a group of states with "similar characteristics"
- Compute a "cost" $\hat{J}(j)$ for each aggregate state $S_j$
- Approximate the optimal cost of each original system state $x$ with $\tilde{J}(x) = \sum_j w_{xj} \hat{J}(j)$
- For each x, the $w_{xj}, j = 1, \ldots, m$, are the "aggregation probabilities" ... roughly the degrees of membership of state $x$ in the aggregate states $S_j$

# Hard Aggregation v.s. Soft Aggregation

- Hard aggregation: each original state belongs to one and only one $S_j$, i.e., $w_{xj} \in \{0, 1\}$
- Soft aggregation: $0 \leq w_{xj} \leq 1, \sum_j w_{xj} = 1$




Hard Aggregation




Soft Aggregation

# Example: Discretization

- Consider a problem with the state space being a bounded region of a Euclidean space
- Discretize using some finite grid $\{x^1, \cdots, x^M\}$
- Any nongrid state can be expressed as

$$x = \sum_{m=1}^{M} w^m(x) x^m$$

with aggregation probability $w^m(x)$ satisfying $0 \leq w^m(x) \leq 1$, $\sum_{m=1}^{M} w^m(x) = 1$

- The costs-to-go of any state is approximated by

$$\tilde{J}(x) = \sum_{m=1}^{M} w^m(x) \hat{J}(x^m)$$

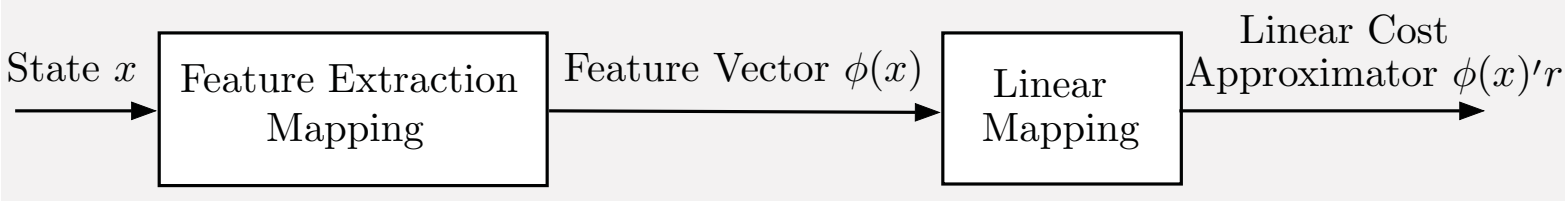
# Parametric Cost-to-Go Approximation

- Use a cost-to-go approximation from a parametric class $\tilde{J}(x, r)$ where $x$ is the current state and $r = (r_1, ..., r_m)$ is a vector of "tunable" scalars (weights)
- By adjusting the weights, one can change the "shape" of the approximation $\tilde{J}$ so that it is reasonably close to the true optimal cost-to-go function
- Two key issues:
  - The choice of parametric class $\tilde{J}(x, r)$ (the approximation architecture)
  - Method for tuning the weights ("training" the architecture)
- Successful application strongly depends on how these issues are handled, and on insight about the problem.

# Approximation Architecture

- Linear or nonlinear approximation w.r.t. the dependence of $\tilde{J}(x, r)$ on $r$
- Linear architectures are easier to train, but non-linear ones (e.g., neural networks) are richer
- Linear feature-based architecture: $\phi = (\phi_1, \ldots, \phi_n)$

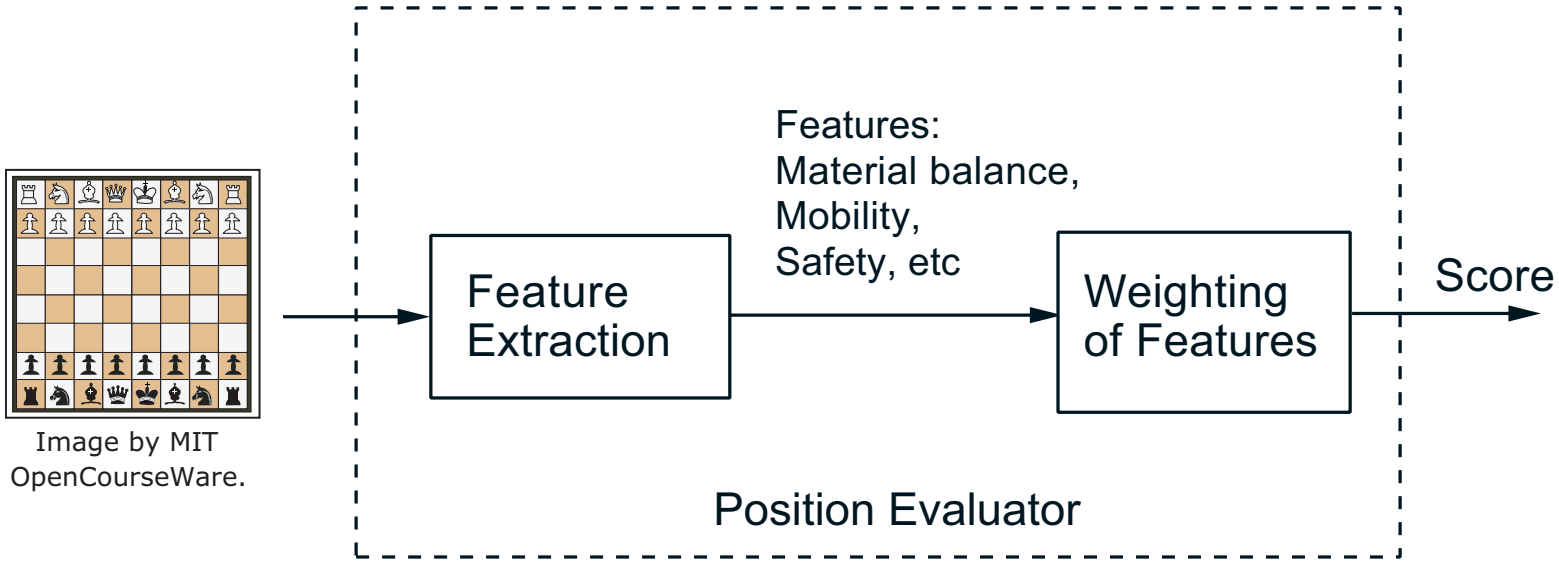
$$\tilde{J}(x, r) = \phi' r = \sum_j \phi_j(x) r_j$$




- Ideally, the features will encode much of the nonlinearity that is inherent in the cost-to-go approximated, and the approximation may be quite accurate without a complicated architecture

# Computer Chess Example

- Chess programs use a feature-based position evaluator that assigns a score to each move/position




- Many context-dependent special features
- Most often the weighting of features is linear but multistep lookahead is involved
- Most often the training is done "manually", by trial and error

# Rollout Algorithms

- Rollout algorithm: When $\tilde{J}_k$ is the cost-to-go of some heuristic policy (called the base policy)
- Policy iteration for infinite horizon problem is rollout
- Policy improvement property: The rollout algorithm achieves no worse (and usually much better) cost than the base heuristic starting from the same state
- Main difficulty: Calculating $\tilde{J}_k(x_k)$ maybe computationally intensive if the cost-to-go of the base policy cannot be analytically calculated
  - May involve Monte Carlo simulation if the problem is stochastic

# Example: The Quiz Problem

- A list of $N$ questions
- If question $i$ is answered correctly (with prob. $p_i$), we receive a reward $R_i$; if not, the quiz terminates.
- Objective: choose order of questions to maximize expected reward
- Optimal policy (index policy): Questions should be answered in decreasing order of $p_i R_i/(1-p_i)$

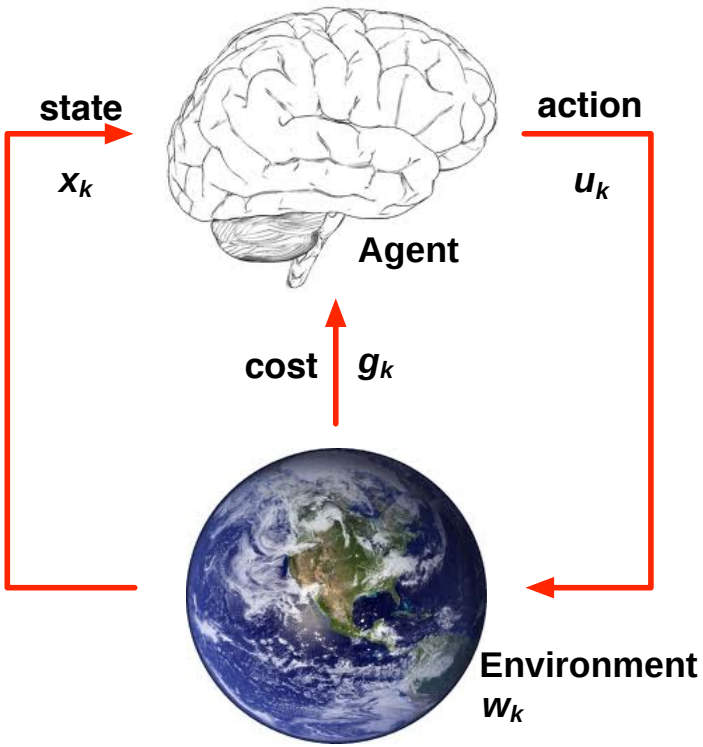
# Example: The Quiz Problem

- With minor changes in the problem, the index policy need not be optimal
- Example: A limit ($< N$) on the maximum number of questions that can be answered
  - If only one question can be answered, it is optimal to answer the one that offers the maximum expected reward $p_i R_i$
- Use index policy as the base policy for rollout: easy to calculate

# Outline


1. Introduction
2. Approximate Dynamic Programming
3. Reinforcement Learning

# RL Problem




- The environment is initially (partially) unknown
- The agent interacts with the environment and improves its policy
- Or the model is known but overcomplex (a.k.a. ADP)

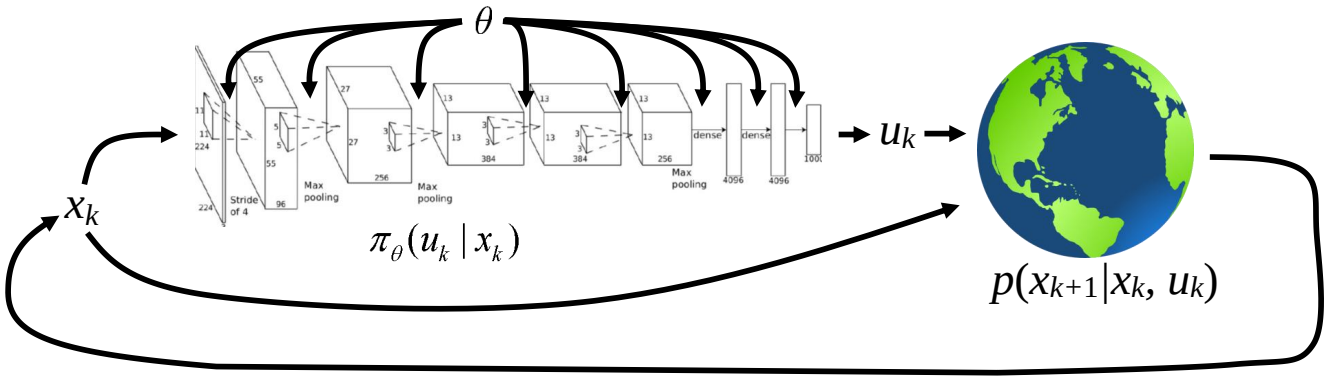
# RL Problem

- Objective

$$\min_{\pi} \lim_{N\to\infty} E_{(x_k,u_k)\sim p_\theta(x_k,u_k)} \left\{ \sum_{k=0}^{N-1} \alpha^k g_k(x_k, u_k) \right\}$$

where $0 < \alpha \leq 1$

- The policy can be randomized $\pi(u_k|x_k) \in [0,1]$
- Parametric policy representation $\pi_\theta(u_k|x_k)$




  - Learn the parameter $\theta$ instead of $\pi$ (Deep Reinforcement Learning)

# RL Problem

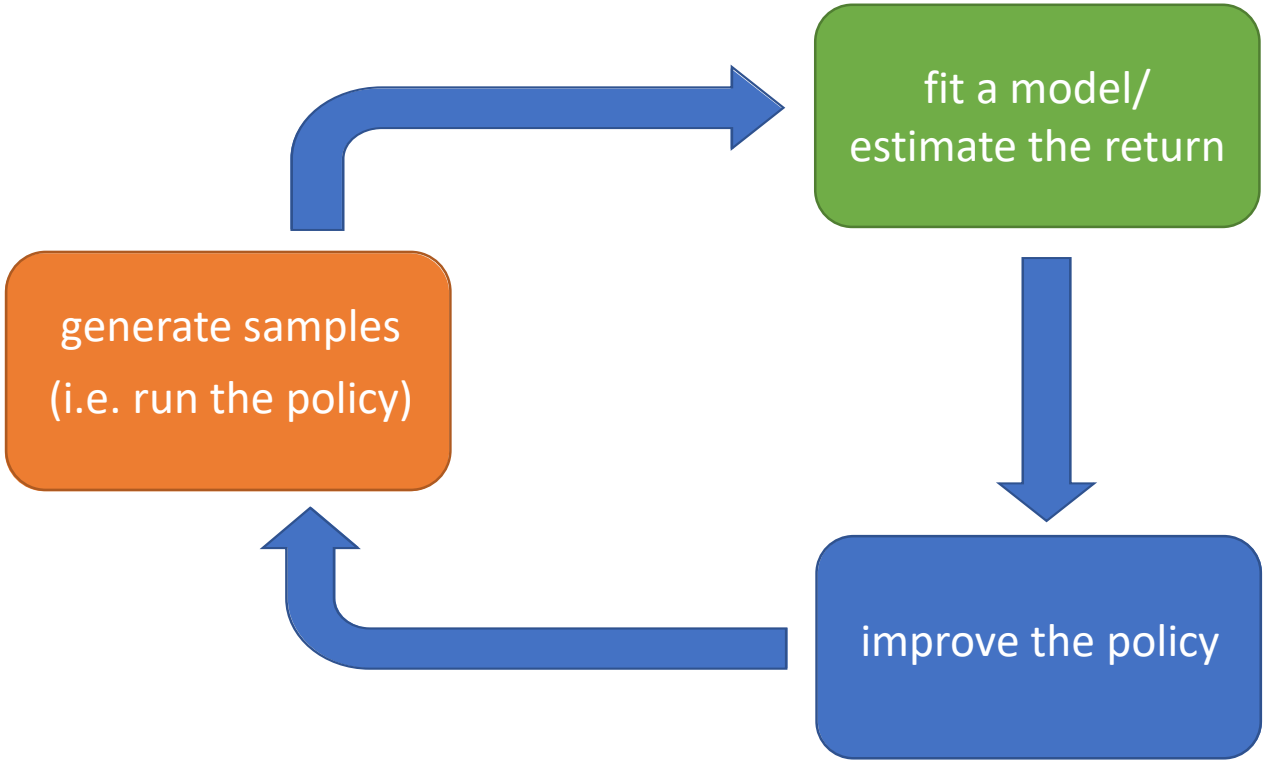
- Objective

$$\min_{\pi} \lim_{N\to\infty} E_{(x_k,u_k)\sim p_\theta(x_k,u_k)} \left\{ \sum_{k=0}^{N-1} \alpha^k g_k(x_k,u_k) \right\}$$

  where $0 < \alpha \leq 1$
- The expectation is taken over all possible state-action pair transitions with prob. $p(x_{k+1}|x_k,u_k)$ under policy $\pi_\theta(x_k,u_k)$
  - Calculate the expectation without knowledge of $p(x_{k+1}|x_k,u_k)$
  - Transition probability changes as policy changes
- State $x_k$ maybe not known, only some observation $o_k$, a.k.a., partially observed Markov decision process (POMDP)
- Cost function $g_k$ maybe hard to model, or not explicitly additive
  - E.g., go game

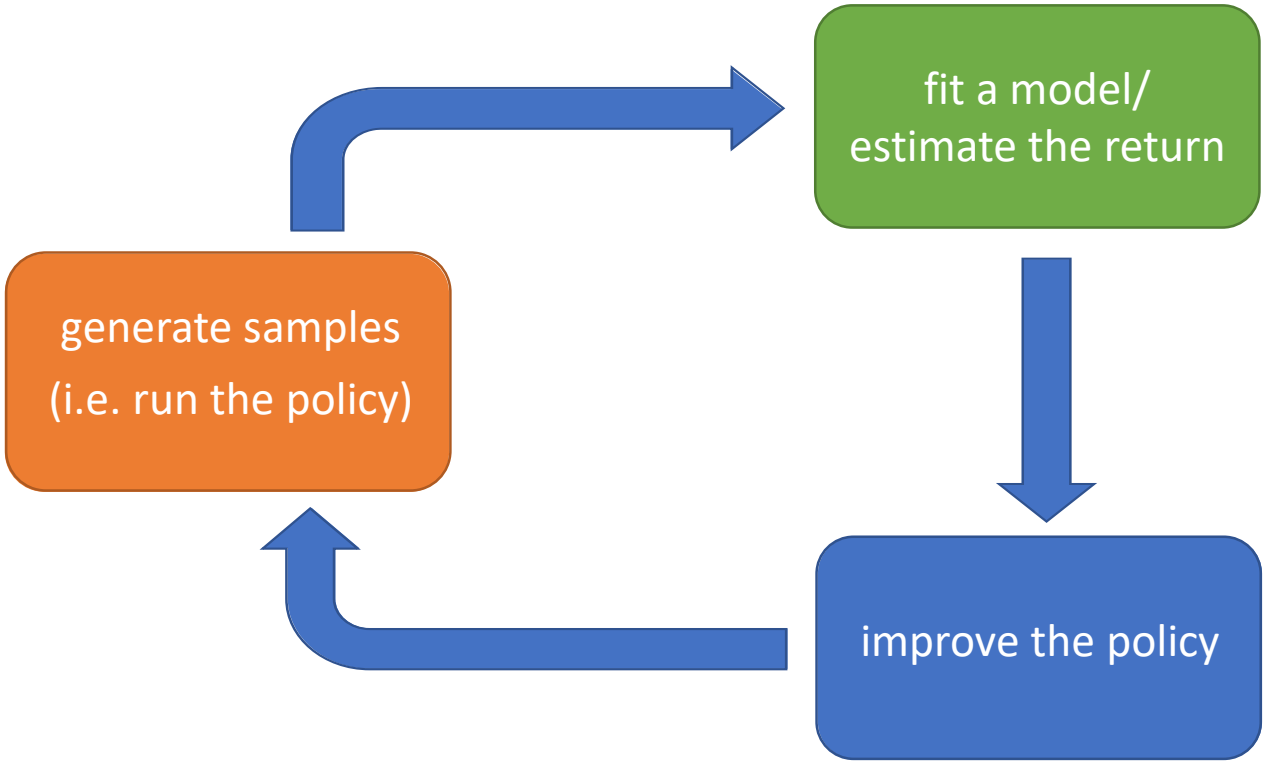
# Framework

- Basic idea: iteratively improve policy like DP
- A key issue: estimate expectations based on samples
- Tradeoff: accuracy v.s. efficiency



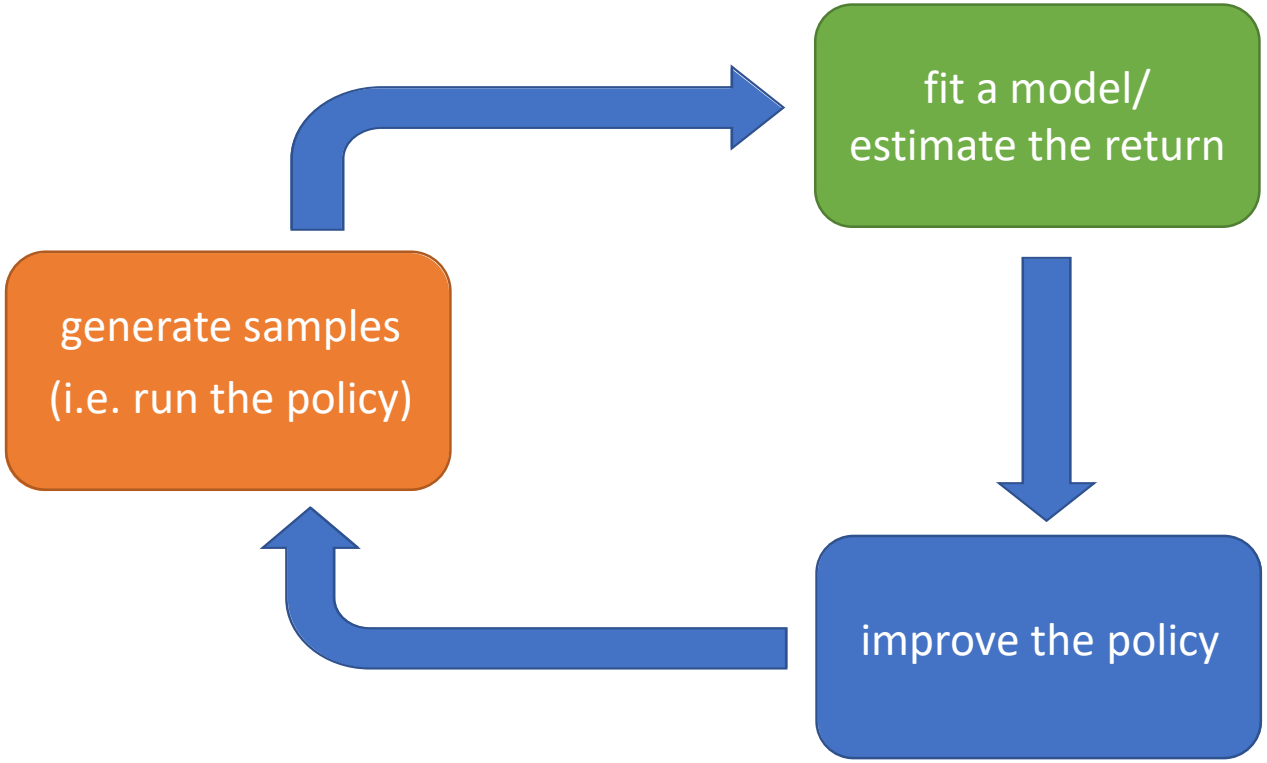
# Example - Model-based

- Generate $N$ samples with $\pi_\theta$, each with sufficiently long stages
- Fit a model $f_\phi$ such that $x_{k+1} \approx f_\phi(x_k, u_k)$
- Improve Policy: train $\pi_\theta(x_k) = u_k$ based on the model $f_\phi$ and $g$



# Example - Model-free

- Generate $N$ samples with $\pi_\theta$, each with sufficiently long stages
- Estimate return as $J(\theta) = E_{\pi_\theta}\left[\sum_k \alpha^k g_k\right] \approx \frac{1}{N}\sum_{i=1}^{N}\sum_k \alpha^k g_k$
- Improve Policy: $\theta \leftarrow \theta + \gamma \bigtriangledown_\theta J(\theta)$



# Model-based v.s. Model-free

- Model-based RL
  - Learn the model $p(x_{k+1}|x_k, u_k), g_k(x_k, u_k)$ from experience
  - Plan value/policy from model (solve an MDP)
  - Advantage: Can explicitly describe model uncertainty
  - Disadvantage:
    - Two-fold estimation errors
    - Unapplicable if the model is too large
- Model-free RL
  - Directly learn value/policy from experience
  - Application cases:
    - MDP model is unknown, but experience can be sampled
    - MDP model is known, but to big to use

# Definitions

- Q-function: total cost from taking $u$ in $x$ following $\pi$

$$Q_\pi(x,u) = \sum_{k=0}^{N-1} E_\pi[\alpha^k g_k(x_k,u_k)|x_0 = x, u_0 = u]$$

- Value function: total cost starting from $x$ following $\pi$

$$V_\pi(x) = \sum_{k=0}^{N-1} E_\pi[\alpha^k g_k(x_k,u_k)|x_0 = x]$$

- $V_\pi(x) = E_{u\sim\pi(u|x)}[Q_\pi(x,u)]$
- RL problem: $\min_\pi E_{x\sim p(x)}[V_\pi(x)]$

# Ways to Improve Policy

- Idea 1: if we know $Q_\pi(x, u)$ for policy $\pi$, we can improve $\pi$:
  set $\pi'(u|x) = 1$ if $u = \arg\min_u Q_\pi(x, u)$
  this policy is at least as good as $\pi$ (and probably better)!
  and it doesn't matter what $\pi$ is

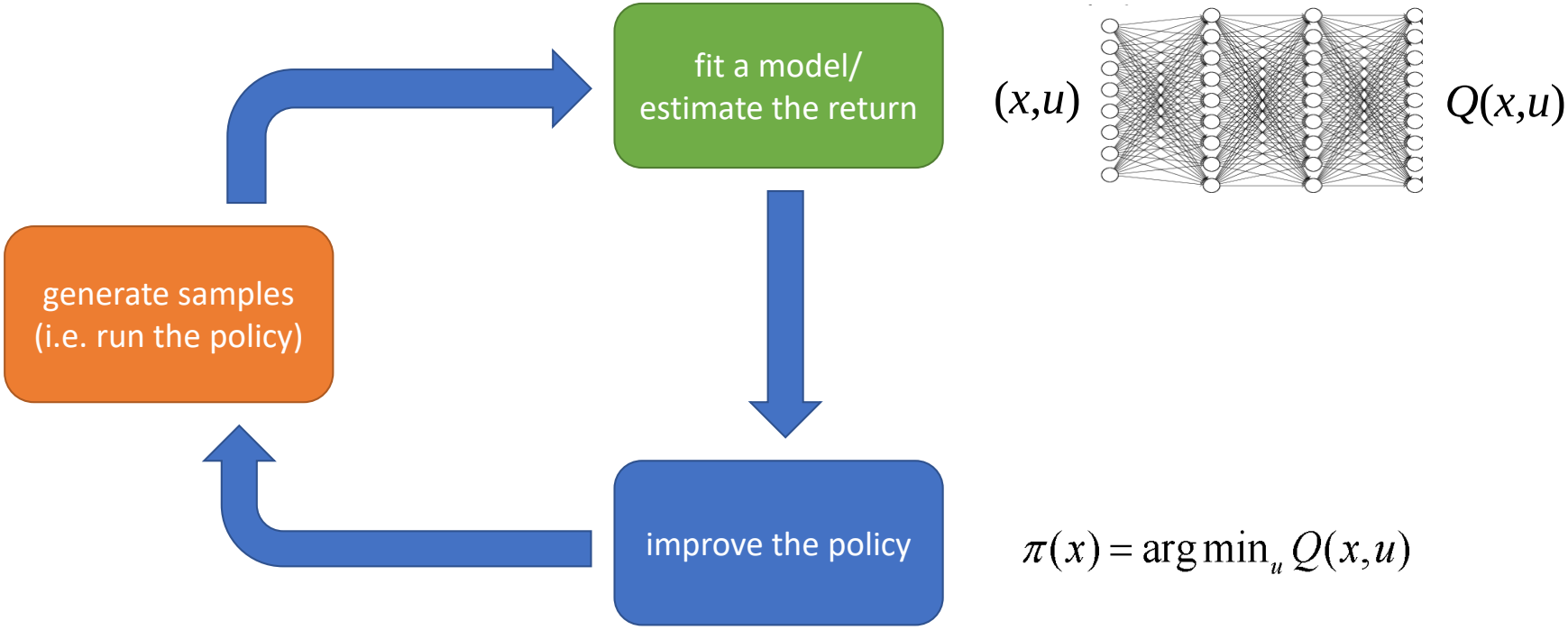
- Idea 2: compute gradient to increase probability of good actions $u$:
  recall that $V_\pi(x) = E_{u\sim\pi(u|x)}[Q_\pi(x, u)]$ under policy $\pi(u|a)$
  if $Q_\pi(x, u) < V_\pi(x)$, $u$ is better than average
  modify $\pi(u|x)$ to increase probability of $u$ if $Q_\pi(x, u) < V_\pi(x)$

# Types of Reinforcement Learning

- Value-based: estimate value function or Q-function of the optimal policy (no explicit policy)
- Policy-based: directly differentiate the objective with policy gradient
- Actor-critic: estimate value function or Q-function of the current policy, use it to improve policy
- Model-based: estimate the transition model, and then use it for planning

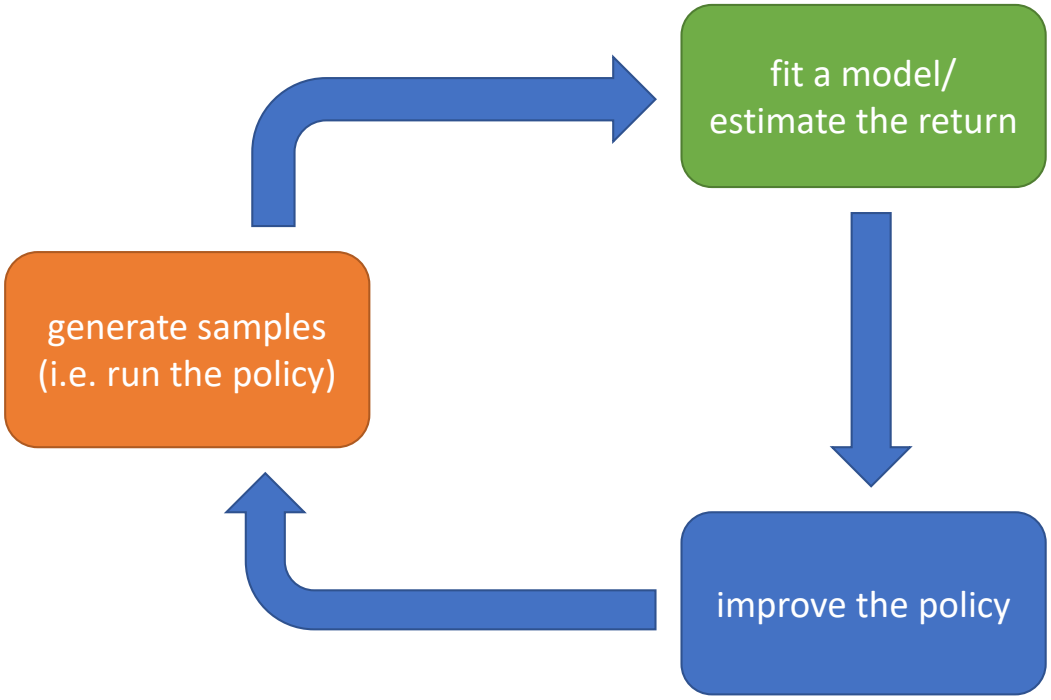
# Value-based Algorithms

- Generate samples with $\pi$
- Fit $V(x)$ or $Q(x,u)$
- Set $\pi(x) = \arg\min_u Q(x,u)$

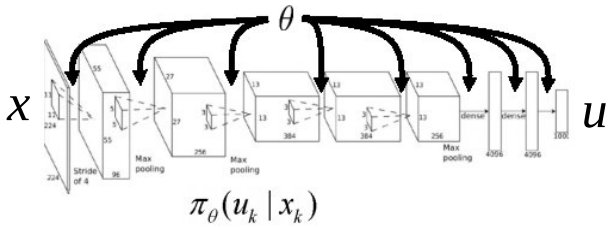


# Policy-based Algorithms

- Generate samples with $\pi_\theta$
- Evaluate returns $J(\theta) = \sum_k \alpha^k g_k(x_k, u_k)$
- Improve policy $\theta \leftarrow \theta + \gamma \bigtriangledown_\theta J(\theta)$



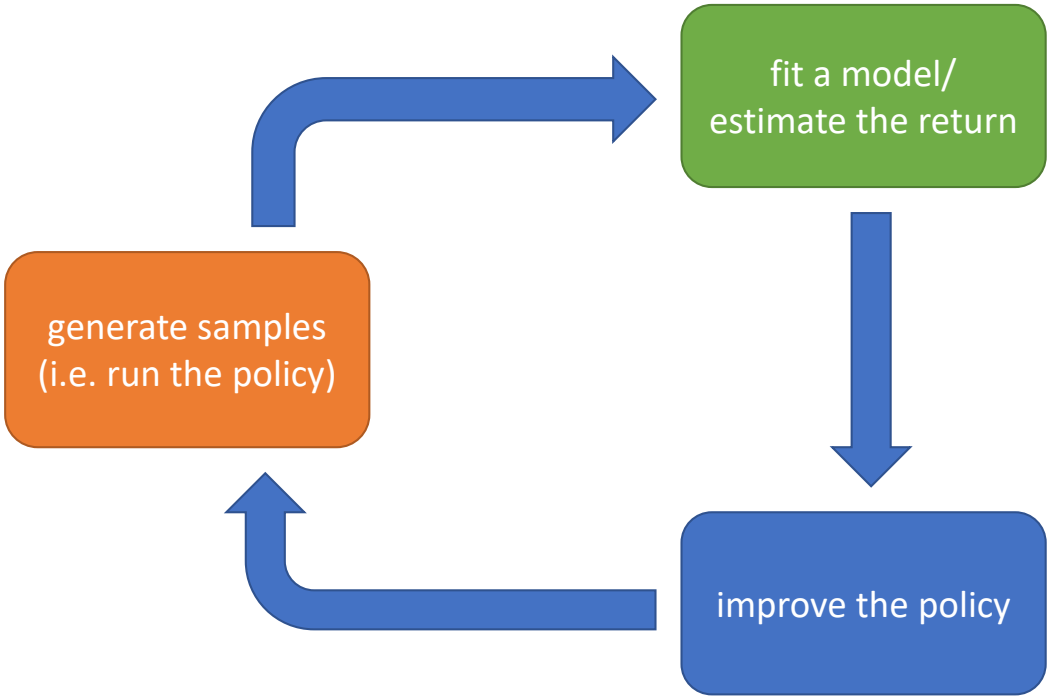

$$J(\theta) = \sum_k \alpha^k g_k(x_k, u_k)$$

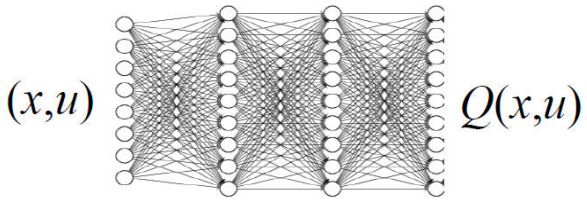


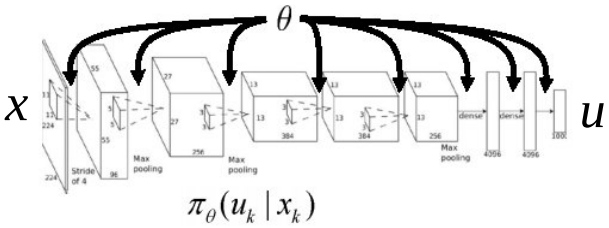

$$\theta \leftarrow \theta + \gamma \bigtriangledown_\theta J(\theta)$$

# Actor-Critic Algorithms

- Generate samples with $\pi_\theta$
- Fit $V(x)$ or $Q(x,u)$
- Improve policy $\theta \leftarrow \theta + \gamma \bigtriangledown_\theta E[Q(x,u)]$



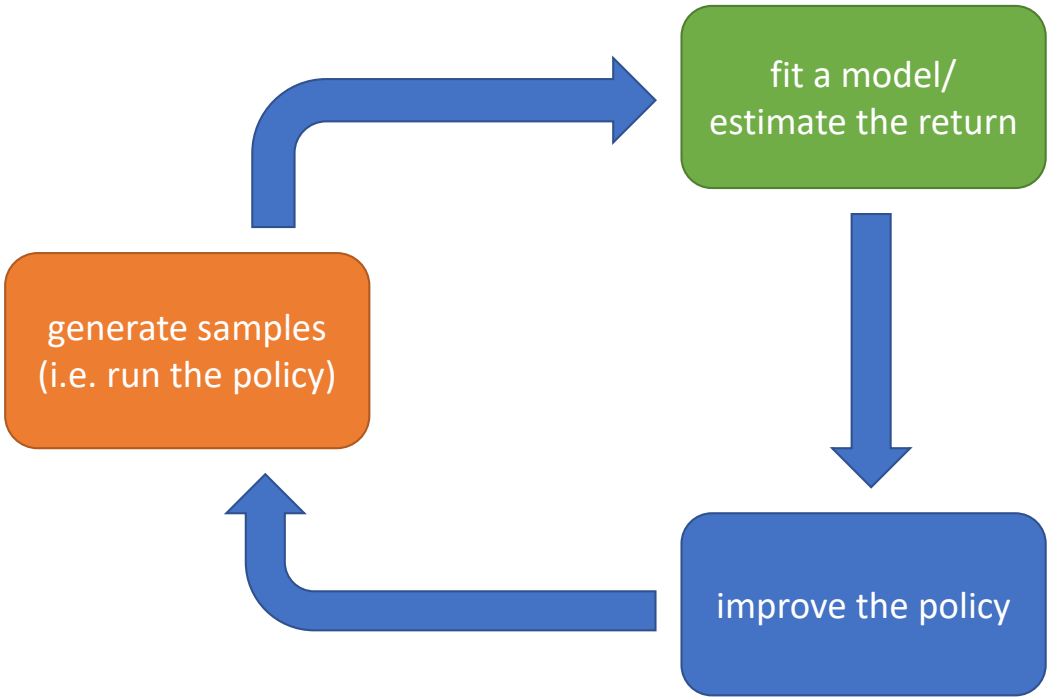

$$\theta \leftarrow \theta + \gamma \bigtriangledown_\theta J(\theta)$$

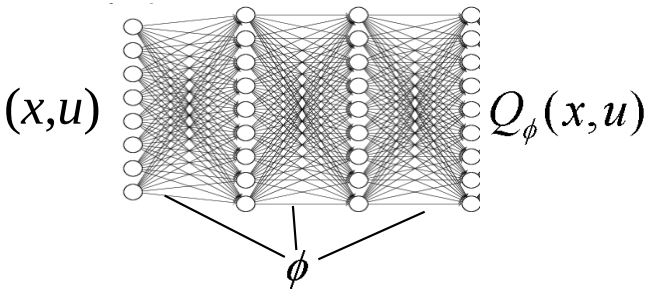
# Examples of Specific Algorithms

- Value-based
  - Q-learning, DQN
  - Temporal difference learning
  - Fitted value iteration
- Policy-based
  - Policy gradient
  - Trust region policy optimization
- Actor-critic
  - Asynchronous advantage actor-critic (A3C)
  - Soft actor-critic (SAC)
- Model-based
  - Dyna
  - Guided policy search

# Q-learning

- Collect dataset $\{(x_i, u_i, x_i', g_i)\}$ using policy $\pi$
- Set $\tilde{Q}_i \leftarrow g_i(x_i, u_i) + \alpha \max_{u'} Q_\phi(x_i', u_i')$
- Set $\phi \leftarrow \arg\min_\phi \sum_i ||Q_\phi(x_i, u_i) - \tilde{Q}_i||^2$

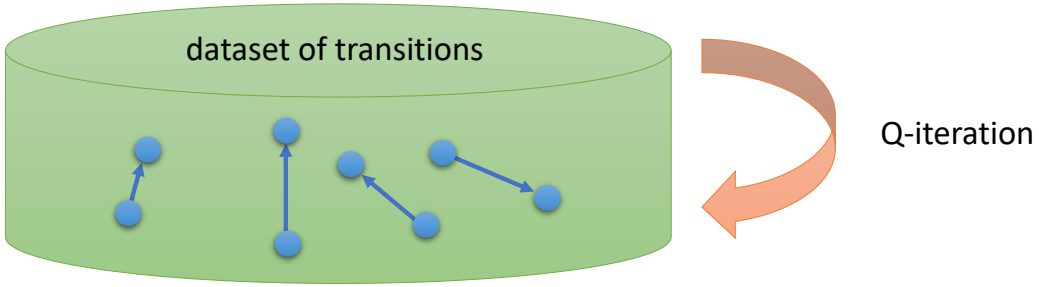



$$\pi(x) = \arg\min_u Q(x,u)$$

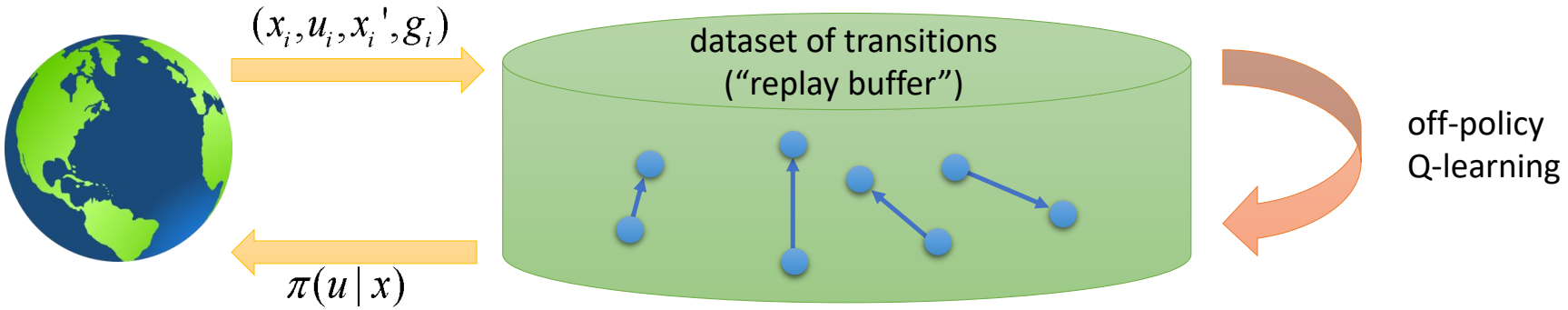
# Replay Buffer

- Problem: sequential states are strongly correlated
- Observation: Data from any policy will work!
- Solution: replay buffer



# Q-learning with Replay Buffer

- Collect dataset $\{(x_i, u_i, x_i', g_i)\}$ using policy $\pi$, add to replay buffer
- Load data from buffer, set $\tilde{Q}_i \leftarrow g_i(x_i, u_i) + \alpha \max_{u'} Q_\phi(x_i', u_i')$
- Set $\phi \leftarrow \arg\min_\phi \sum_i ||Q_\phi(x_i, u_i) - \tilde{Q}_i||^2$



# Exploration v.s. Exploitation

- Reinforcement learning is like trial-and-error learning
- Tradeoff between exploration and exploitation
  - Restaurant Selection: go to your favourite restaurant, or try a new restaurant
- **Exploitation** exploits known information to minimize cost

$$\pi(x) = \arg\min_u Q(x, u)$$

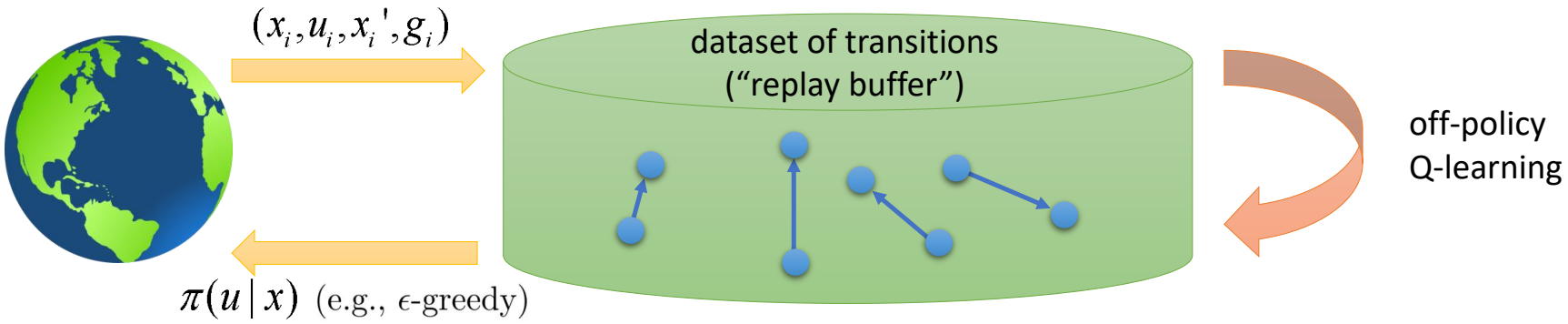
- **Exploration** finds more information about the environment
  - Example: randomly select a policy
- The agent should discover a good policy from its experiences of the environment without losing too much reward along the way

# $\epsilon$-greedy

- With probability $1-\epsilon$ to update policy by

$$\pi(x) = \arg\min_u Q(x,u)$$

- With probability $\epsilon$ to randomly select a policy



# RL v.s. ML

Two fundamental problems in learning

- Reinforcement Learning
  - Data (rewards) maybe sparse, noisy and delayed
  - Data samples are highly correlated
  - Data distribution changes as new behaviours are learned
- Machine Learning
  - Require large amount labelled training data
  - Data samples are assumed independent
  - Data distribution is fixed